Load balancer

Research by Adan Abo Mokh , Alaa Abo Mokh, Mayyas Iktilat
Advisor Eran Tavor

INTRODUCTION

Load balancing distributes incoming client requests among several backend servers to avoid overloading a single server. In this project, we compare IP Hash, Least Connections, and Least Connections – Second Tie to analyze how load balancing decisions affect system performance.

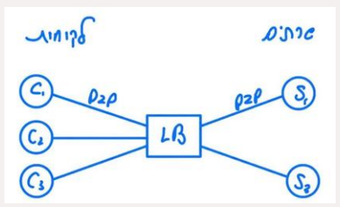
OBJECTIVE

This project aims to compare IP Hash, Least Connections, and Least Connections — Second Tie under different server and traffic conditions. We focus on how server speed, client mapping, request timing, and tie-breaking affect Total Completion Time (TCT).

In our research, we focused on two main questions:
1. How do IP Hash and Least Connections perform under symmetric and heterogeneous server conditions?
2. How does the tie-breaking rule in Least Connections affect performance when the servers have different speeds?

METHODOLOGY

We built a Mininet-based topology with three clients, one load balancer, and two backend servers: h3 and h4. The clients sent different request types to the load balancer, and each request was forwarded to one of the backend servers according to the selected load balancing algorithm. We tested IP Hash, Least Connections, and Least Connections — Second Tie under several conditions, including symmetric servers, heterogeneous servers, different request gaps, and different client-to-server mappings. The main metric used for evaluation was Total Completion Time (TCT), and the experiments were repeated to obtain more stable and reliable results.



Load Balancing Algorithms — Overview

- **IP Hash:** A static algorithm that maps each client IP address to a fixed backend server. It provides consistent routing, but it does not check the current server load. Its performance depends strongly on how clients are mapped to the servers.
- **Least Connections:** A dynamic algorithm that chooses the server with the fewest active connections. It reacts to the current state of the system and can avoid sending more requests to an already busy server.
- **Least Connections — Second Tie:** A modified version of Least Connections. When both servers have the same number of active connections, it chooses the second server instead of the first one. In our final scenario, this improved performance because the second server was faster.

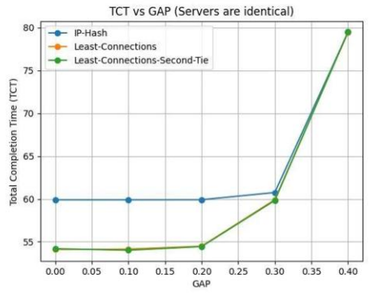
OBSERVATIONS

1. When the servers were identical, the three algorithms produced very close results. Since both backend servers had the same processing speed, the tie-breaking rule did not create a strong advantage for one server over the other. This shows that when the system is balanced, the choice of algorithm has a smaller effect on Total Completion Time.

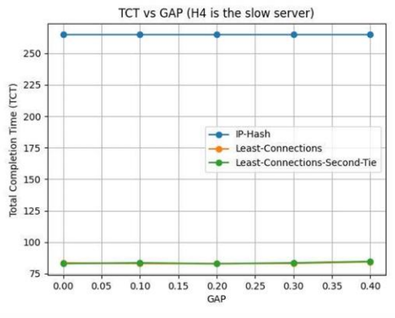
2. When h4 was the slow server, IP Hash performed poorly because the client-to-server mapping sent more traffic to h4. In contrast, Least Connections and Least Connections — Second Tie achieved much lower Total Completion Time because they reacted to active connections and avoided overloading the slower server. This result shows that request distribution becomes critical when backend servers have different speeds.

3. This mapping shows why IP Hash is sensitive to the client distribution. In our setup, two clients were mapped to h4 and only one client was mapped to h3. As a result, IP Hash sends a larger part of the traffic to h4 regardless of the current server load. Therefore, the performance of IP Hash depends heavily on whether h4 is the fast or slow server.

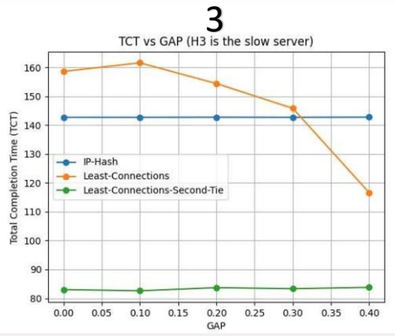
1



2



3



CONCLUSIONS

- When the servers are similar, the difference between IP Hash and Least Connections is small.
- When one server is slower, request distribution becomes much more important because sending too many requests to the slow server increases the Total Completion Time.
- Least Connections usually performs better in heterogeneous setups because it reacts to the number of active connections.
- IP Hash can perform well only when the fixed client mapping happens to send more traffic to the faster server.
- Tie-breaking is an important design choice. In our final scenario, Least Connections Second Tie achieved the best result because it sent more traffic to the faster server during ties.